

456. 132 Pattern

Solved 

Medium

Topics

Companies

Given an array of n integers `nums`, a **132 pattern** is a subsequence of three integers `nums[i]`, `nums[j]` and `nums[k]` such that $i < j < k$ and $nums[i] < nums[k] < nums[j]$.

Return `true` if there is a **132 pattern** in `nums`, otherwise, return `false`.

Example 1:

Input: `nums = [1,2,3,4]`

Output: `false`

Explanation: There is no 132 pattern in the sequence.

Example 2:

Input: `nums = [3,1,4,2]`

Output: `true`

Explanation: There is a 132 pattern in the sequence: `[1, 4, 2]`.

Example 3:

Input: `nums = [-1,3,2,0]`

Output: `true`

Explanation: There are three 132 patterns in the sequence: `[-1, 3, 2]`, `[-1, 3, 0]` and `[-1, 2, 0]`.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 2 * 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

```
class Solution {
    public boolean find132pattern(int[] nums) {
        Deque<Integer> stack = new ArrayDeque<>();
        int second = Integer.MIN_VALUE; // this is the "2" in 132
        for (int i = nums.length - 1; i >= 0; i--) {
            // nums[i] acts as "1"
            if (nums[i] < second) {
                return true;
            }
            // nums[i] acts as a new "3"
            while (!stack.isEmpty() && nums[i] > stack.peekLast()) {
                second = stack.pollLast(); // popped value becomes best "2"
            }
            stack.addLast(nums[i]);
        }
        return false;
    }
}
```

→ Set second = min of integer.

→ for loop from left to right.

$O(N)$

* if stack top is $<$ $nums[i]$, then pop until the condition fails & assign it to second.

* Once we get second value, which means we already have third value. So if we get any value that is less than second we get our first number.

* add every element to stack at end.

* if $nums[i] <$ second & second is not null, then return true as it is only true if second is updated & second updates only when third is found.

→ Return false at last.